```
import os
import random
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from PIL import Image

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
from sklearn.metrics import classification_report
```

Task 1: Data Understanding and Visualization:

1. Load and visualize images from a dataset stored in directories, where each subdirec- tory represents a class.

```
train_dir = "/content/drive/MyDrive/AI L6/FruitinAmazon/FruitinAmazon/train"
test_dir  = "/content/drive/MyDrive/AI L6/FruitinAmazon/FruitinAmazon/test"

class_names = sorted(os.listdir(train_dir))
print(f"Found {len(class_names)} classes: {class_names}")
```

```
Found 6 classes: ['acai', 'cupuacu', 'graviola', 'guarana', 'pupunha', 'tucuma']
```

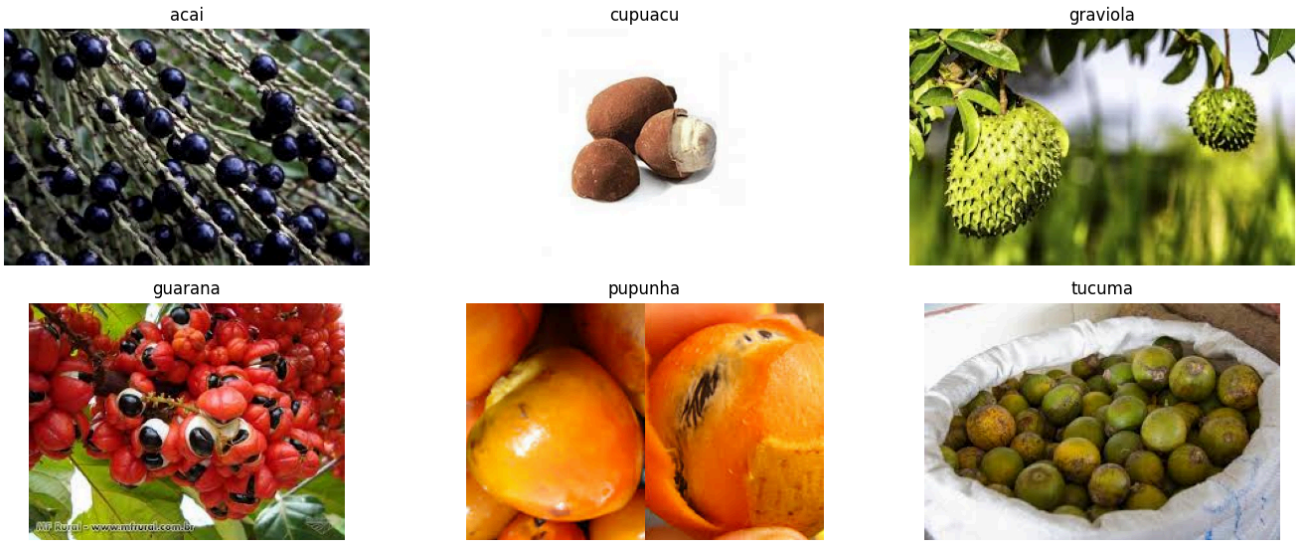
```
selected_images = []
selected_labels = []

for class_name in class_names:
    class_path = os.path.join(train_dir, class_name)
    if os.path.isdir(class_path):
        images = [img for img in os.listdir(class_path)
                  if img.lower().endswith(('.png', '.jpg', '.jpeg'))]
        if images:
            selected_img = os.path.join(class_path, random.choice(images))
            selected_images.append(selected_img)
            selected_labels.append(class_name)

num_classes = len(selected_images)
cols = (num_classes + 1) // 2
rows = 2

fig, axes = plt.subplots(rows, cols, figsize=(14, 6))
for i, ax in enumerate(axes.flat):
    if i < num_classes:
        img = mpimg.imread(selected_images[i])
        ax.imshow(img)
        ax.set_title(selected_labels[i], fontsize=12)
        ax.axis("off")
    else:
        ax.axis("off")

plt.suptitle("One Sample Image Per Class", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



2. Check for Corrupted Image:

```
corrupted_images = []

for class_name in class_names:
    class_path = os.path.join(train_dir, class_name)
    if os.path.isdir(class_path):
        for img_name in os.listdir(class_path):
            img_path = os.path.join(class_path, img_name)
            try:
                with Image.open(img_path) as img:
                    img.verify()
            except (IOError, SyntaxError):
                corrupted_images.append(img_path)
                os.remove(img_path)
                print(f"Removed corrupted image: {img_path}")

if not corrupted_images:
    print("No corrupted images found.")
else:
    print(f"\nTotal corrupted images removed: {len(corrupted_images)}")
```

```
No corrupted images found.
```

Task 2: Loading and Preprocessing Image Data in keras:

```
img_height     = 128
img_width      = 128
batch_size     = 16
validation_split = 0.2

rescale = tf.keras.layers.Rescaling(1.0 / 255)

train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    train_dir,
    labels='inferred',
    label_mode='int',
    image_size=(img_height, img_width),
    interpolation='nearest',
    batch_size=batch_size,
    shuffle=True,
    validation_split=validation_split,
    subset='training',
    seed=123
)
train_ds = train_ds.map(lambda x, y: (rescale(x), y))

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
```

```
    train_dir,
    labels='inferred',
    label_mode='int',
    image_size=(img_height, img_width),
    interpolation='nearest',
    batch_size=batch_size,
    shuffle=False,
    validation_split=validation_split,
    subset='validation',
    seed=123
)
val_ds = val_ds.map(lambda x, y: (rescale(x), y))

print("Training and Validation datasets loaded successfully.")
```

```
Found 90 files belonging to 6 classes.
Using 72 files for training.
Found 90 files belonging to 6 classes.
Using 18 files for validation.
Training and Validation datasets loaded successfully.
```

Task 3: Implement the CNN Model

Architecture as specified in the worksheet:

- Conv(32, 3x3, same) → ReLU → MaxPool(2x2)
- Conv(32, 3x3, same) → ReLU → MaxPool(2x2)
- Flatten → Dense(128) → Dense(64) → Dense(num_classes)

```
num_classes = len(class_names)
print(f"Number of classes: {num_classes}")

model = keras.Sequential([

    layers.Conv2D(32, (3, 3), padding='same', activation='relu',
                  input_shape=(img_height, img_width, 3)),
    layers.MaxPooling2D((2, 2), strides=2),

    layers.Conv2D(32, (3, 3), padding='same', activation='relu'),
    layers.MaxPooling2D((2, 2), strides=2),

    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),

    layers.Dense(num_classes, activation='softmax')
])

model.summary()
```

```
Number of classes: 6
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 128, 128, 32)	896
max_pooling2d_2 (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_3 (Conv2D)	(None, 64, 64, 32)	9,248
max_pooling2d_3 (MaxPooling2D)	(None, 32, 32, 32)	0
flatten_1 (Flatten)	(None, 32768)	0
dense_3 (Dense)	(None, 128)	4,194,432
dense_4 (Dense)	(None, 64)	8,256
dense_5 (Dense)	(None, 6)	390

```
Total params: 4,213,222 (16.07 MB)
Trainable params: 4,213,222 (16.07 MB)
```

Task 4: Compile and train the Model

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
print("Model compiled.")
```

```
Model compiled.
```

```
checkpoint_cb = ModelCheckpoint(
    'best_model_ws5.h5',
    monitor='val_accuracy',
    save_best_only=True,
    verbose=1
)

early_stop_cb = EarlyStopping(
    monitor='val_loss',
    patience=20,
    restore_best_weights=True,
    verbose=1
)

# Train
history = model.fit(
    train_ds,
    epochs=250,
    batch_size=16,
    validation_data=val_ds,
    callbacks=[checkpoint_cb, early_stop_cb]
)
```

```
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 517ms/step - accuracy: 1.0000 - loss: 4.2094e-04 - val_accuracy: 0.8333 - val_loss: 0.3849
Epoch 26/250
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 402ms/step - accuracy: 1.0000 - loss: 3.1161e-04
Epoch 26: val_accuracy did not improve from 0.88889
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 457ms/step - accuracy: 1.0000 - loss: 3.7811e-04 - val_accuracy: 0.8333 - val_loss: 0.3921
Epoch 27/250
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 282ms/step - accuracy: 1.0000 - loss: 4.2584e-04
Epoch 27: val_accuracy did not improve from 0.88889
5/5 ━━━━━━━━━━━━━━━━━━━━ 4s 334ms/step - accuracy: 1.0000 - loss: 3.5237e-04 - val_accuracy: 0.8333 - val_loss: 0.4189
Epoch 27: early stopping
Restoring model weights from the end of the best epoch: 7.
```

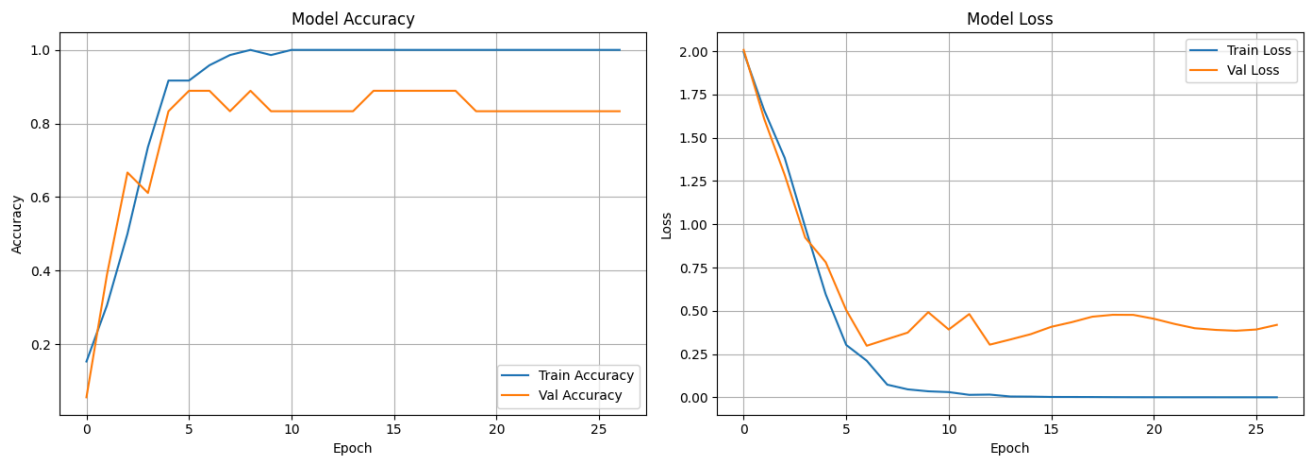
Training / Validation Loss & Accuracy Curves

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(history.history['accuracy'],     label='Train Accuracy')
axes[0].plot(history.history['val_accuracy'], label='Val Accuracy')
axes[0].set_title('Model Accuracy')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Accuracy')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(history.history['loss'],     label='Train Loss')
axes[1].plot(history.history['val_loss'], label='Val Loss')
axes[1].set_title('Model Loss')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.savefig('training_curves_ws5.png', dpi=150)
plt.show()
```



Task 5: Evaluate the Model

```
test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    test_dir,
    labels='inferred',
    label_mode='int',
    image_size=(img_height, img_width),
    interpolation='nearest',
    batch_size=batch_size,
    shuffle=False
)
test_ds = test_ds.map(lambda x, y: (rescale(x), y))

test_loss, test_acc = model.evaluate(test_ds)
print(f"\nTest Loss     : {test_loss:.4f}")
print(f"Test Accuracy: {test_acc:.4f}")
```

```
Found 30 files belonging to 6 classes.
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - accuracy: 0.6333 - loss: 0.9328

Test Loss    : 0.9328
Test Accuracy: 0.6333
```

Task 6: Save and Load the Model

```
model.save('cnn_fruits_ws5.h5')
print("Model saved to cnn_fruits_ws5.h5")

loaded_model = keras.models.load_model('cnn_fruits_ws5.h5')
print("Model loaded successfully.")

loaded_loss, loaded_acc = loaded_model.evaluate(test_ds)
print(f"\nLoaded Model Test Loss    : {loaded_loss:.4f}")
print(f"Loaded Model Test Accuracy: {loaded_acc:.4f}")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file forma
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty u
Model saved to cnn_fruits_ws5.h5
Model loaded successfully.
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 376ms/step - accuracy: 0.6333 - loss: 0.9328

Loaded Model Test Loss    : 0.9328
Loaded Model Test Accuracy: 0.6333
```

Task 7: Predictions and Classification Report

```
y_true = []
y_pred = []

for images, labels in test_ds:
    preds = loaded_model.predict(images, verbose=0)
    y_pred.extend(np.argmax(preds, axis=1))
    y_true.extend(labels.numpy())

print("\nClassification Report")
print(classification_report(y_true, y_pred, target_names=class_names))
```

```
Classification Report
              precision    recall  f1-score   support

        acai       0.57      0.80      0.67         5
     cupuacu       0.50      1.00      0.67         5
    graviola       1.00      0.40      0.57         5
     guarana       0.80      0.80      0.80         5
     pupunha       1.00      0.40      0.57         5
      tucuma       0.50      0.40      0.44         5

    accuracy                           0.63        30
   macro avg       0.73      0.63      0.62        30
weighted avg       0.73      0.63      0.62        30
```